

Chapter 1 Exam Notes: Introduction and Operating-System Structure

Modern Operating Systems Study Notes

Contents

1	Scope	2
2	1.3 Computer Hardware Review	2
2.1	Processor Concepts	2
2.2	Memory Hierarchy	2
2.3	Disk and I/O Hardware	2
2.4	Booting	3
3	1.4 Operating-System Zoo: Definitions Only	3
4	1.5 Core Concepts to Remember Briefly	3
5	1.6 System Calls: Call and Description	4
5.1	Process Management Calls	4
5.2	File and Directory Calls: Brief	4
5.3	Miscellaneous Calls	4
5.4	System-Call Flow	4
6	1.7 Operating-System Structure	5
6.1	Monolithic System	5
6.2	Layered System	5
6.3	Microkernel	5
6.4	Client-Server Model	5
6.5	Virtual Machines	5
6.6	Exokernels	5
7	Likely Short Questions with Answers	5

1 Scope

Exam focus: Start from Section 1.3. Treat 1.3 as generic concepts, 1.4 as definitions only, keep files brief because file systems have a separate chapter, study 1.6 system calls, and understand 1.7 operating-system structures.

2 1.3 Computer Hardware Review

2.1 Processor Concepts

- **CPU:** executes instructions and changes state through registers, program counter, stack pointer, and status register.
- **Program counter:** holds the address of the next instruction to execute.
- **Stack pointer:** points to the current top of the stack, used for procedure calls, local variables, interrupt handling, and returns.
- **User mode:** restricted mode for ordinary programs. Privileged instructions are not allowed.
- **Kernel mode:** privileged mode where the operating system can execute I/O instructions, manage memory, control interrupts, and access protected hardware state.
- **Trap:** a controlled switch from user mode to kernel mode, usually caused by a system call or exception.
- **Interrupt:** an asynchronous signal from hardware, such as disk completion, keyboard input, or timer tick.

2.2 Memory Hierarchy

- Registers are fastest and smallest.
- Cache stores recently used memory blocks to reduce average memory access time.
- Main memory stores active processes and kernel data.
- Disk/SSD stores persistent data and inactive pages/files.
- **Locality:** programs tend to reuse recently accessed data and nearby data. Caches and virtual memory rely on this.

2.3 Disk and I/O Hardware

- Disks are block devices: data is transferred in fixed-size blocks.
- Device controllers hide low-level device details and expose registers or memory-mapped control areas.

- DMA allows a controller to move data between device and memory without CPU copying every word.
- Buses connect CPU, memory, and I/O controllers. Bus bandwidth can become a bottleneck.

2.4 Booting

1. Firmware starts after power-on.
2. Firmware tests hardware and locates a bootable device.
3. A boot loader is loaded.
4. The boot loader loads the operating-system kernel.
5. The kernel initializes memory, devices, interrupts, process tables, and starts system services.

3 1.4 Operating-System Zoo: Definitions Only

Mainframe OS Designed for large I/O-heavy systems and batch/transaction workloads.

Server OS Provides services over a network, such as web, file, database, and authentication services.

Multiprocessor OS Manages multiple CPUs/cores and must handle synchronization and scheduling across them.

Personal-computer OS General-purpose interactive operating system for a single user or small number of users.

Handheld OS Designed for mobile devices with touch UI, sensors, wireless networking, and power limits.

Embedded OS Built into devices such as routers, appliances, cars, and controllers.

Sensor-node OS Very small OS for low-power distributed sensors.

Real-time OS Must respond within timing deadlines. Hard real-time cannot miss deadlines; soft real-time tolerates occasional misses.

Smart-card OS Very small OS for cards with limited CPU, memory, and I/O.

4 1.5 Core Concepts to Remember Briefly

- **Process:** a program in execution, including address space, registers, open files, and scheduling state.
- **Address space:** the memory addresses a process can use.
- **File:** an abstraction for persistent information. Detailed file-system topics are mainly Chapter 4.

- **Directory:** maps names to files or other directories.
- **Shell:** command interpreter that starts programs and connects them with files/devices.

5 1.6 System Calls: Call and Description

5.1 Process Management Calls

Call	Description
<code>fork()</code>	Create a child process as a copy of the calling process.
<code>exec()</code>	Replace the current process image with a new program.
<code>waitpid()</code>	Wait for a child process to terminate and collect its status.
<code>exit()</code>	Terminate the calling process and return status to parent.

5.2 File and Directory Calls: Brief

Call	Description
<code>open()</code>	Open a file and return a file descriptor.
<code>close()</code>	Release an open file descriptor.
<code>read()</code>	Copy bytes from a file/device into a buffer.
<code>write()</code>	Copy bytes from a buffer to a file/device.
<code>lseek()</code>	Change the current file offset.
<code>stat()</code>	Get metadata such as size, type, owner, and timestamps.
<code>mkdir()</code>	Create a directory.
<code>rmdir()</code>	Remove an empty directory.
<code>link()</code>	Create another directory entry for an existing file.
<code>unlink()</code>	Remove a directory entry. File data disappears when no links/open references remain.

5.3 Miscellaneous Calls

- Time calls read or set clocks.
- Protection calls change ownership, permissions, or process identity.
- Signal calls install handlers and control asynchronous notifications.

5.4 System-Call Flow

1. User program places call number and arguments where the ABI requires.
2. A trap instruction switches CPU to kernel mode.
3. Kernel validates arguments and permissions.
4. Kernel performs the requested service.
5. Kernel returns result/error code and switches back to user mode.

6 1.7 Operating-System Structure

6.1 Monolithic System

Most OS services run in one large kernel address space. Fast calls between components, but bugs can affect the whole kernel.

6.2 Layered System

The OS is divided into layers. Each layer uses only lower layers. Easier reasoning, but strict layering can add overhead or force awkward interfaces.

6.3 Microkernel

Keep only minimal mechanisms in kernel mode: low-level address spaces, IPC, basic scheduling. File systems, drivers, and services may run in user mode. Better isolation, but IPC overhead and design complexity matter.

6.4 Client-Server Model

Processes request services from server processes through messages. This structure is natural for distributed systems and microkernels.

6.5 Virtual Machines

A hypervisor provides each guest OS with a virtual hardware interface. Useful for isolation, consolidation, testing, and compatibility.

6.6 Exokernels

The kernel securely multiplexes hardware resources but leaves abstractions to user-level libraries. Gives flexibility but requires more complex application/library support.

7 Likely Short Questions with Answers

Question

Q1. Difference between user mode and kernel mode

Answer

User mode is the restricted CPU mode used by ordinary programs. In user mode, a program cannot directly execute privileged instructions, control I/O devices, or access protected kernel memory. Kernel mode is the privileged mode used by the operating system. In kernel mode, the OS can manage memory, perform I/O, handle interrupts, and control hardware. The separation protects the system from faulty or malicious user programs.

Question

Q2. Difference between trap, interrupt, and system call

Answer

A trap is a synchronous transfer to the kernel caused by the currently running instruction, such as an exception or system-call instruction. An interrupt is asynchronous and is caused by external hardware, such as a timer or disk controller. A system call is a controlled request made by a user program to ask the OS for a service; it is commonly implemented using a trap.

Question

Q3. Why are system calls needed?

Answer

User programs cannot safely access hardware or protected kernel data directly. System calls provide a controlled interface for services such as process creation, file I/O, memory management, communication, and protection changes. The kernel validates arguments and permissions before performing the operation.

Question

Q4. Write the steps of booting

Answer

After power-on, firmware starts and checks hardware. It locates a bootable device, loads a boot loader, and the boot loader loads the OS kernel. The kernel initializes memory, interrupts, device drivers, process tables, and system services, then starts user-level initialization programs.

Question

Q5. Define the OS types from Section 1.4

Answer

Mainframe OS handles large I/O-heavy batch or transaction workloads. Server OS provides network services. Multiprocessor OS manages multiple CPUs or cores. Personal-computer OS supports interactive general-purpose use. Handheld OS is designed for mobile devices and power limits. Embedded OS runs inside dedicated devices. Sensor-node OS is a very small OS for distributed sensors. Real-time OS gives deadline-based responses. Smart-card OS runs on highly limited smart-card hardware.

Question

Q6. Compare monolithic kernel, layered system, and microkernel

Answer

A monolithic kernel places most OS services inside one kernel address space, making internal calls fast but failures dangerous. A layered system divides the OS into layers where each layer uses lower layers, improving structure but sometimes adding overhead or awkward boundaries. A microkernel keeps only minimal mechanisms in kernel mode and moves services such as file systems or drivers to user mode, improving isolation but increasing IPC cost and design complexity.